

CSC3209 Software architecture and Design Patterns

Lecture6: Relations Between Patterns and Tactics

Subject Lecturer:
Dr. Muhammed Basheer Jasser

References

- Materials from
 - Chapter 13: Len Bass, Paul Clements, Rick Kazman, Software Architecture in Practice, 3/E

Lecture Outline

- Recap from Lecture4
 - What is a Pattern?
 - Pattern Catalogue
 - Module patterns
 - Component and Connector Patterns
 - Allocation Patterns
- Relation Between Tactics and Patterns
- Using tactics together
- Summary

Relationships Between Tactics and Patterns

Patterns Comprise Tactics

- Patterns are built from tactics; if a pattern is a molecule, a tactic is an atom.
- The layered includes several tactics: increase semantic coherence, abstract common services, encapsulate, restrict communication paths, and use an intermediary
 - **Increase semantic coherence.** The goal of ensuring that a layer's responsibilities all work together without excessive reliance on other layers is achieved by choosing responsibilities that have semantic coherence. Doing so binds responsibilities that are likely to be affected by a change.
 - For example, responsibilities that deal with hardware should be allocated to a hardware layer and not to an application layer; a hardware responsibility typically does not have semantic coherence with the application responsibilities.

Relationships Between Tactics and Patterns

Patterns Comprise Tactics

- Patterns are built from tactics; if a pattern is a molecule, a tactic is an atom.
- The layered includes several tactics: increase semantic coherence, abstract common services, encapsulate, restrict communication paths, and use an intermediary
 - **Restrict dependencies.** Layers define an ordering and only allow a layer to use the services of its adjacent lower layer.
 - The possible communication paths are reduced to the number of layers minus one. This limitation has a great influence on the dependencies between the layers and makes it much easier to limit the side effects of replacing a layer.
- Without any one of its tactics, the pattern might be ineffective.

Relationships Between Tactics and Patterns

Patterns Comprise Tactics

- Without any one of its tactics, the pattern might be ineffective.
- If the restrict dependencies tactic is not employed, then any function in any layer can call any other function in any other layer, destroying the low coupling that makes the layering pattern effective.
- If the increase semantic coherence tactic is not employed, then functionality could be randomly sprinkled throughout the layers, destroying the separation of concerns, and hence ease of modification, which is the prime motivation for employing layers in the first place.

Using Tactics to Augment Patterns

- A **pattern** is described as a **solution** to a **class** of **problems** in a **general context**.
- **When** a **pattern** is **chosen** and applied, the **context** of its **application** becomes very **specific**.
- To **make** a **pattern work** in a given **architectural context**, we need to **examine** it from **two perspectives**:
 - The **inherent quality attribute tradeoffs** that the pattern makes. **Patterns** exist to **achieve certain quality attributes**, and we **need** to **compare** the **ones** they **promote** (and the ones they **diminish**) with our needs.
 - **Other quality attributes** that the pattern **isn't directly concerned** with, **but** which it nevertheless **affects**, and which are **important in our application**.

Using Tactics to Augment Patterns

- Patterns solve a specific problem but are neutral or have weaknesses with respect to other qualities.
- To illustrate these concerns, and how to use tactics to augment patterns in general, consider the broker pattern
 - Using this pattern, a client requesting some information from a server does not need to know the location or APIs of the server. The client simply contacts the broker.
- Weaknesses of the Broker Pattern
 - Availability.
 - Performance.
 - Testability.
 - Security.
- Next, we will discuss those weaknesses in more details and how tactics help to avoid them.

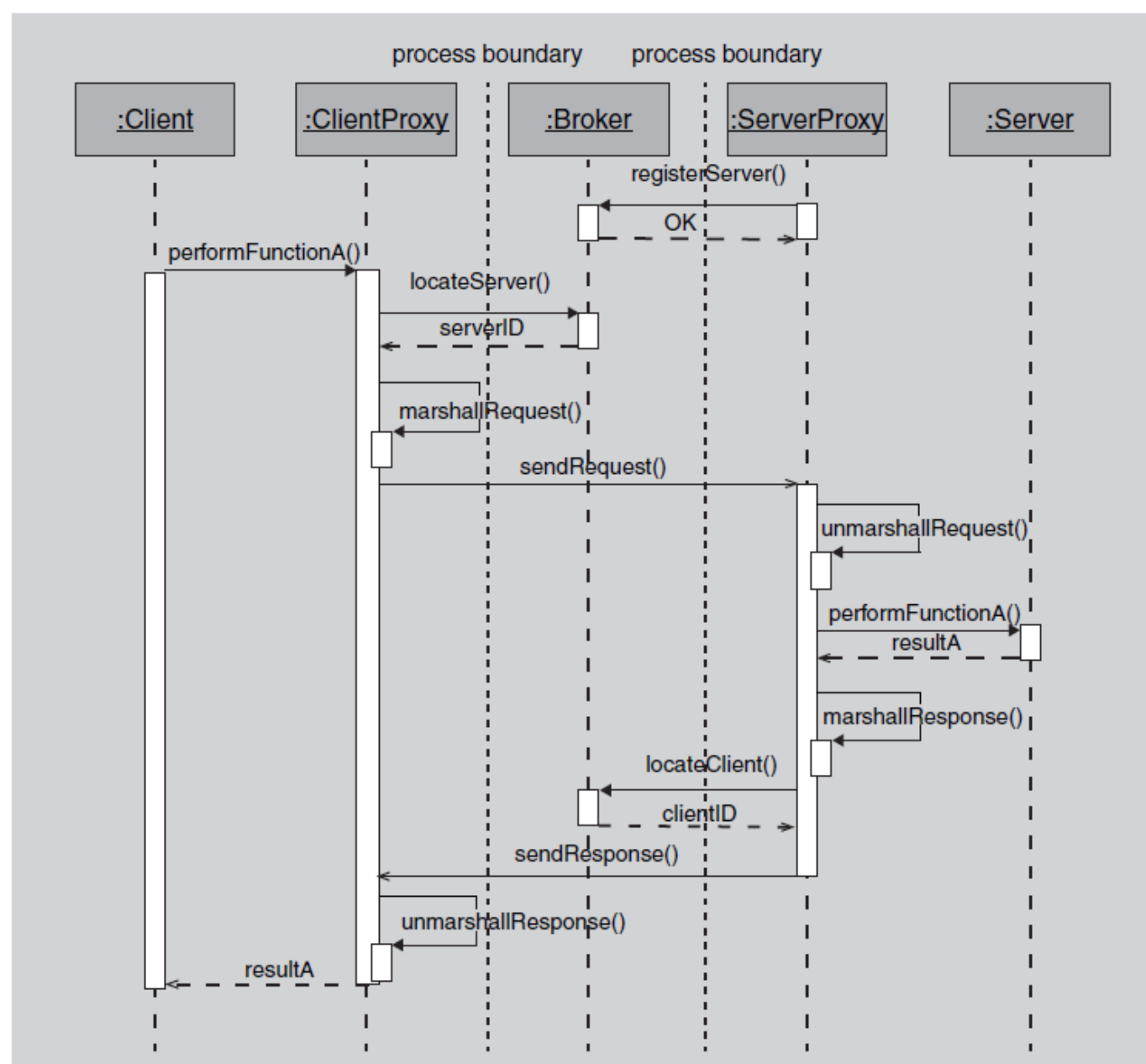


FIGURE 13.16 A sequence diagram showing a typical client-server interaction mediated by a broker

Using Tactics to Augment Patterns

- **Weaknesses of the Broker Pattern:** Availability, Performance, Testability, Security.
- **Availability:** The **broker**, if implemented as suggested in Figure 13.6, is a **single point of failure**. The **liveness** of servers, the broker, and perhaps even the clients **need to be monitored**, and **repair mechanisms must be provided**.
- **Performance.** The levels of **indirection between** the **client** (requesting the information or service) and the **server** (providing the information or service) **add overhead**, and hence **add latency**. Also, the **broker** is a **potential performance bottleneck**.

Using Tactics to Augment Patterns

- **Weaknesses of the Broker Pattern:** Availability, Performance, Testability, Security.
- **Testability.** Brokers are employed in complex multi-process and multiprocessor systems. Such systems are typically highly dynamic. Requests and responses are typically asynchronous. All of this makes testing and debugging such systems extremely difficult.
- **Security.** The broker pattern as presented does not offer any means to authenticate or authorize clients or servers, and provides no means of protecting the communication between clients and servers. This is why security is a legitimate concern

Improving the Broker Pattern with Tactics

- How can we use tactics to augment the broker pattern to let us meet the requirements of a demanding distributed system? Here are some options:
- The increase available resources performance tactic would lead to multiple brokers, to help with performance and availability.
- The maintain multiple copies tactic would allow each of these brokers to share state, to ensure that they respond identically to client requests.
- Load balancing (an application of the scheduling resources tactic) would ensure that one broker is not overloaded while another one sits idle.
- Heartbeat, exception detection, or ping/echo would give the replicated brokers a way of notifying clients and notifying each other when one of them is out of service, as a means of detecting faults.

Using Tactics Together

- Every tactic has its main effect—to manage modifiability or performance or safety, and so on—and it has its side effects, its tradeoffs.
- We will walk through an example that shows how applying tactics to a pattern can produce negative effects in one area, but how adding other tactics can bring relief and put you back in an acceptable design space.
- The point is to show the interplay between tactics that you can use to your advantage.

Using Tactics Together (Example)

- Consider a **system** that **needs** to **detect faults** in its components.
- A **common tactic** for detecting faults is **ping/echo**. Let us assume that the architect has decided to employ ping/echo as a way to detect failed components in the system.
- Every tactic has one or more **side effects**, and ping/echo is no different.
- **Common considerations** associated with **ping/echo** are these:
 - **Security**. How to **prevent** a **ping flood attack**?
 - **Performance**. How to **ensure** that the **performance overhead** of **ping/echo** is **small**?
 - **Modifiability**. How to **add ping/echo** to the **existing architecture**?

Using Tactics Together (Example)

- We can represent the architect's reasoning and decisions thus far as shown in Figure 13.17.

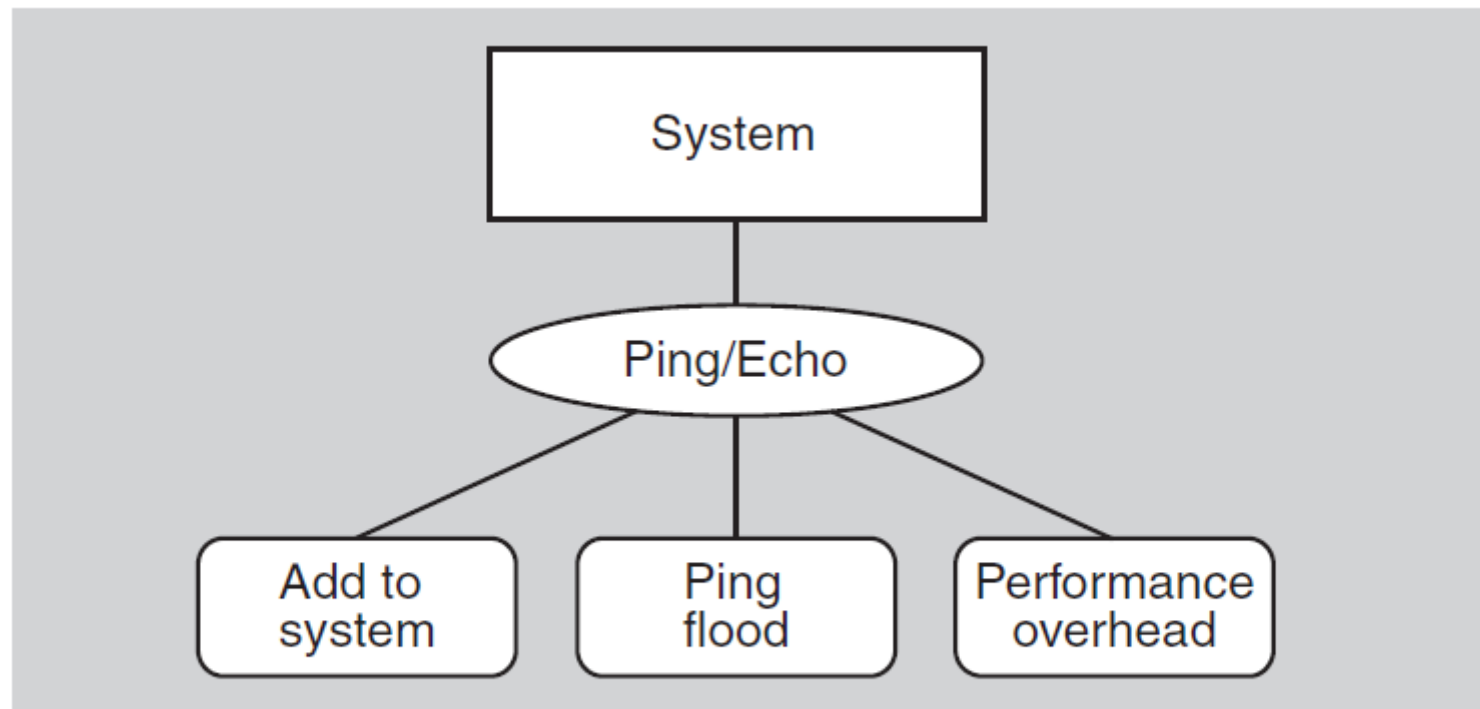


FIGURE 13.17 Partial availability decisions

Using Tactics Together (Example)

- Suppose the architect determines that the performance tradeoff (the overhead of adding ping/echo to the system) is the most severe. A tactic to address the performance side effect is increase available resources.
- Considerations associated with increase available resources are these:
 - Cost. Increased resources cost more.
 - Performance. How to utilize the increased resources efficiently?.
- This set of design decisions can now be represented as shown in Figure 13.18

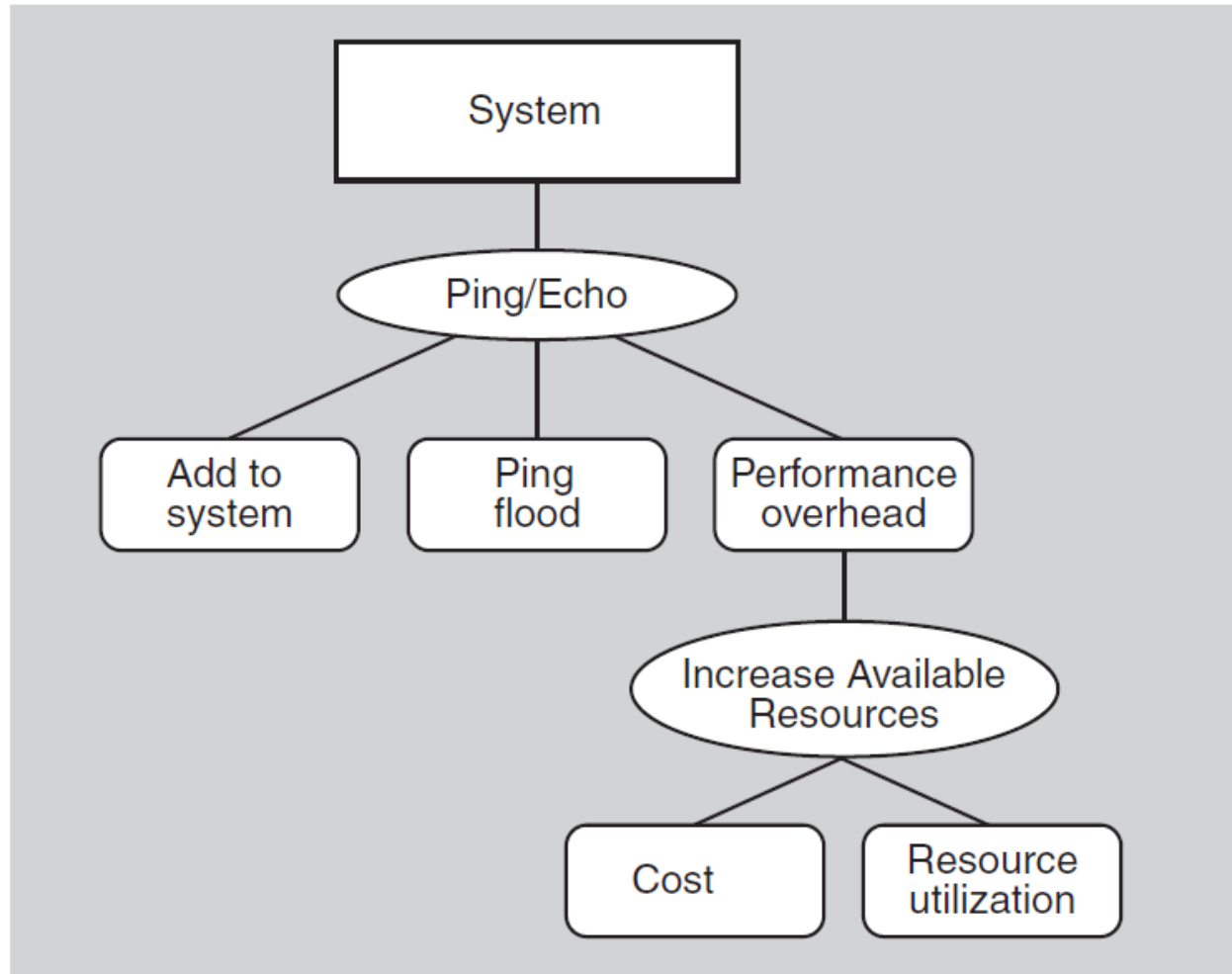


FIGURE 13.18 More availability decisions

Using Tactics Together (Example)

- Now the **architect chooses** to **deal** with the **resource utilization** consequence of employing increase available resources.
- These resources **must be used efficiently** or else they are simply **adding cost** and **complexity** to the system.
- A tactic that can **address** the **efficient use** of **resources** is **the employment of a scheduling policy**.
- **Considerations** associated with the scheduling policy tactic are these:
 - **Modifiability**. How to **add** the **scheduling policy** to the existing architecture?
 - **Modifiability**. How to **change** the **scheduling policy** in the **future**?

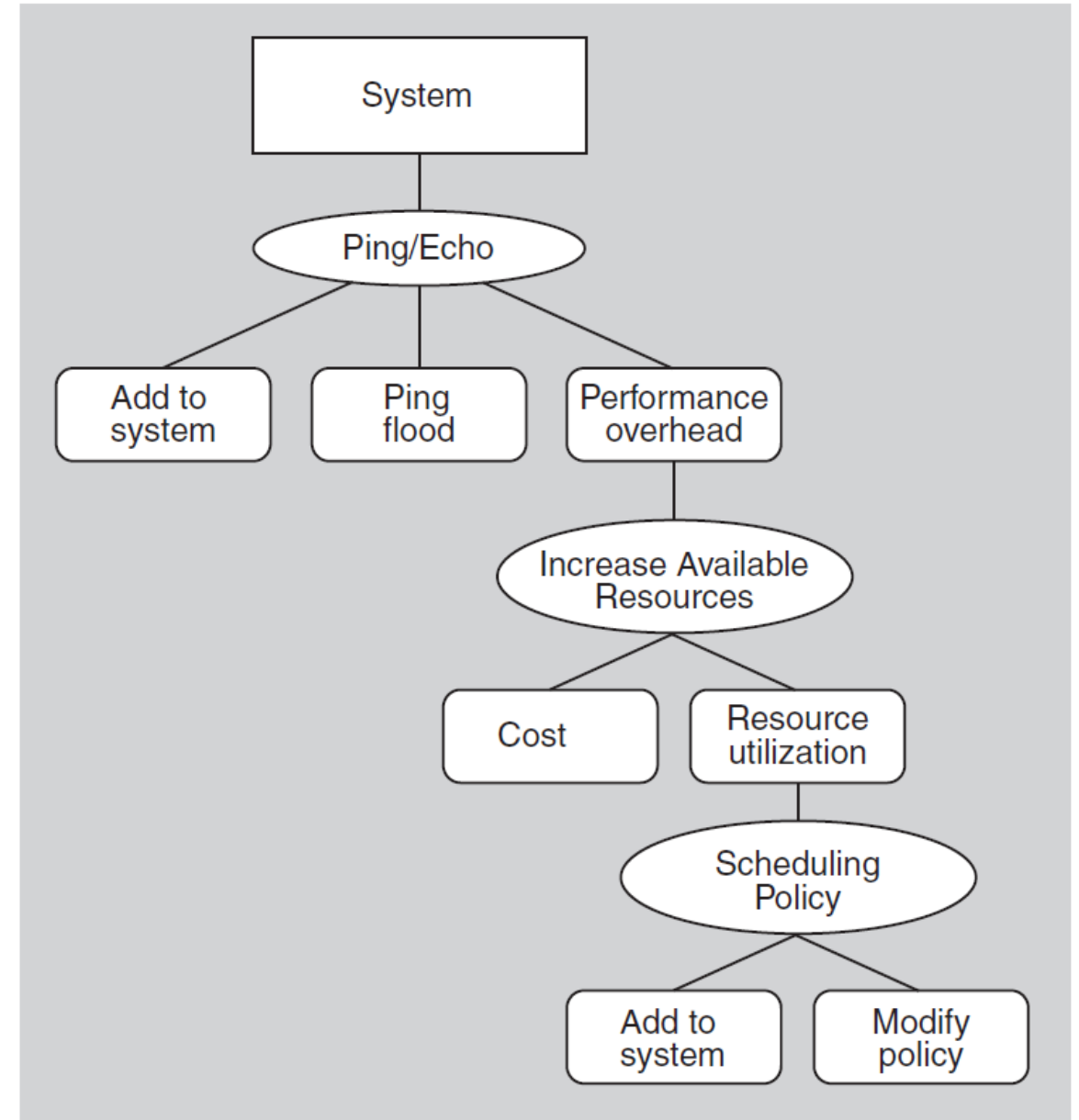
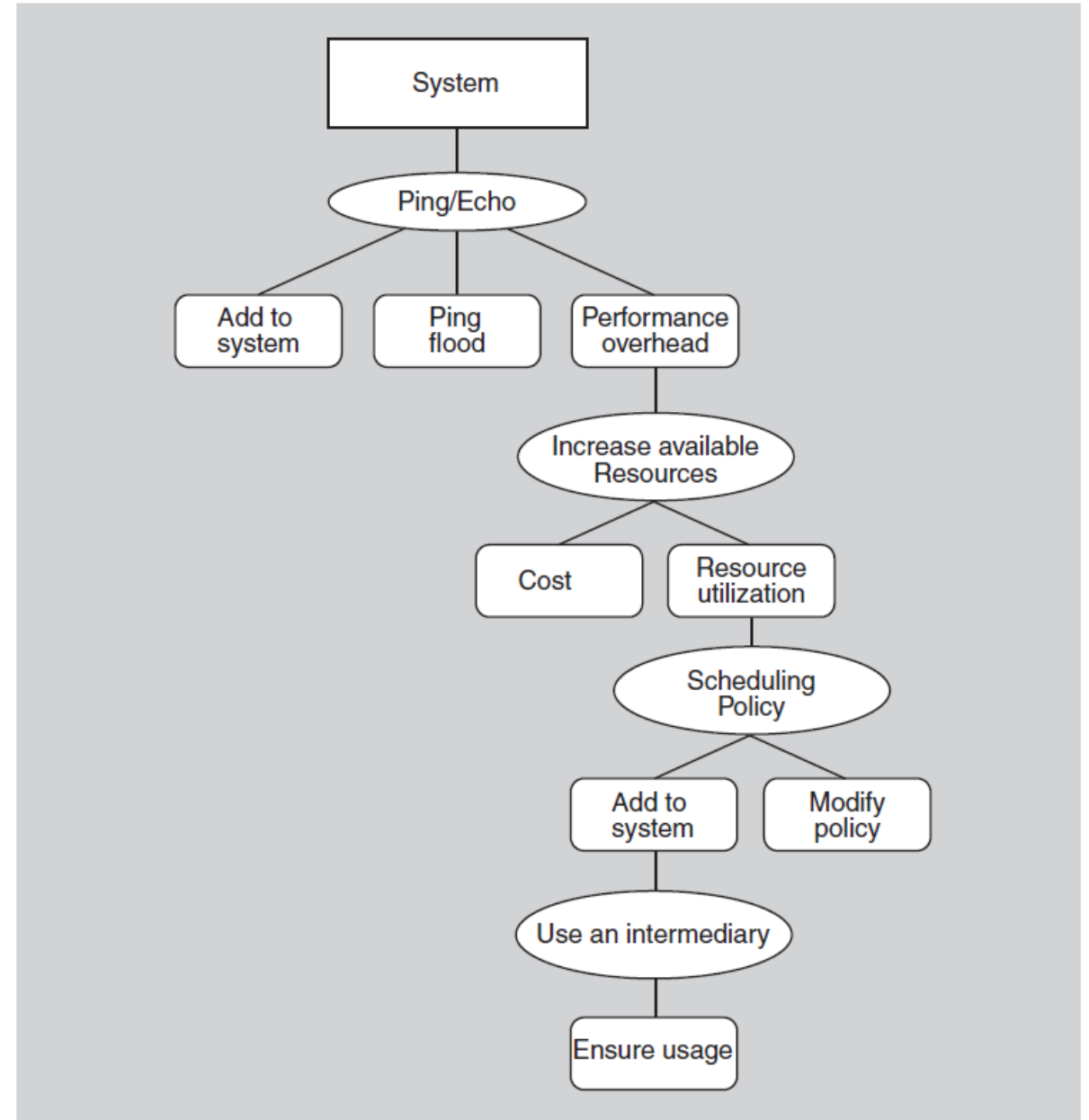


FIGURE 13.19 Still more availability decisions

Using Tactics Together (Example)

- Next the architect chooses to deal with the modifiability consequence of employing a scheduling policy tactic.
- A tactic to address the addition of the scheduler to the system is to use an intermediary, which will insulate the choice of scheduling policy from the rest of the system.
- One consideration associated with use an intermediary is this:
 - Modifiability. How to ensure that all communication passes through the intermediary?



Using Tactics Together (Example)

- A tactic to address the concern that all communication passes through the intermediary is restrict dependencies.
- One consideration associated with the restrict dependencies tactic is this:
 - Performance. How to ensure that the performance overhead of the intermediary is not excessive?

How Does This Process End?

- Each use of tactic introduces new concerns.
- Each new concern causes new tactics to be added.
- This design problem has now become recursive! Are we in an infinite progression?
- No. Eventually the side-effects of each tactic become small enough to ignore.
- At this point (or in fact, at any point in the tree of design decisions that we have described) the architect might determine that the performance overhead of the intermediary is small enough that no further design decisions need to be made.

Summary

- An architectural pattern
 - is a package of design decisions that is found repeatedly in practice,
 - has known properties that permit reuse, and
 - describes a *class* of architectures.
- Tactics are simpler than patterns
- Patterns are underspecified with respect to real systems so they have to be augmented with tactics.
 - Augmentation ends when requirements for a specific system are satisfied.